

Seat Pressure Protocol – Smart Monitoring (SPP-SPM) BLE SERVICE SPECIFICATION

Version 1.0

Contents

Abstract.....	3
I. Introduction	3
II. Related Work.....	3
III. System Architecture	4
IV. Custom Protocol Design	5
V. Implementation.....	6
VI. Experimental Validation	8
VII. Limitations	10
VIII. Conclusions.....	10
IX. References.....	10

Abstract

This paper presents the design and implementation of a custom Bluetooth Low Energy (BLE) communication protocol for reliable multiplexed force sensor monitoring using an ESP32-C3 microcontroller. The system implements a custom GATT service including application-layer reliability mechanisms such as sequence numbering, and CRC-based integrity verification. This protocol guarantees consistent and robust data transmission over BLE while ensuring compatibility with the standard MTU size. Experimental results confirm stable real-time performance.

I. Introduction

Wireless sensor monitoring systems need communication methods that are both reliable and quick. Bluetooth Low Energy (BLE) is a popular choice for energy-efficient wireless technology, especially in embedded systems. However, standard BLE notifications provide limited application-layer structure, which may restrict data traceability and integrity verification in sensing applications. This project addresses these limitations by implementing a lightweight application-layer protocol on top of BLE GATT for multiplexed force-sensitive resistor (FSR) monitoring. The proposed approach enhances standard BLE communication by introducing structured data framing, sequence numbering, and integrity verification, while maintaining low overhead and real-time performance.

II. Related Work

Bluetooth Low Energy (BLE) has become a widely adopted wireless technology in embedded and Internet-of-Things (IoT) systems due to its low power consumption and native support for short-range communication [1], [2]. BLE-based sensor monitoring architectures commonly rely on GATT notifications for periodic data transmission, particularly in wearable and biomedical applications [3].

However, standard BLE notifications provide limited application-layer structure. Although reliability is handled at the Link Layer through automatic retransmissions, successful delivery is not explicitly confirmed or tracked at the application level [1], [3]. In applications requiring data traceability, real-time monitoring, or post-processing analysis, this limitation may introduce challenges in ensuring consistent and interpretable data streams.

To address these limitations, several approaches have been proposed in embedded systems, including the use of structured data framing, sequence numbering strategies, and lightweight integrity verification mechanisms implemented at the firmware level [4]. These techniques aim to improve data traceability and robustness while preserving the low overhead characteristics of BLE communication.

The work presented in this paper follows this design philosophy by implementing a lightweight application-layer protocol over BLE GATT notifications. The proposed solution enhances standard BLE communication by introducing a compact frame structure,

sequence-based tracking, and CRC-based integrity verification, while maintaining compatibility with the default MTU configuration and minimizing computational overhead.

III. System Architecture

The system consists of an ESP32-C3 microcontroller connected to two force-sensitive resistors (FSRs) through a digital multiplexing scheme. Two control lines select the active sensor channel, and the analog signal is sampled using the internal ADC. The peripheral operates as a BLE device exposing a custom GATT service.

A. Hardware Architecture

The ESP32-C3 controls two digital lines (CONTROL_A and CONTROL_B) to sequentially select each sensor. Analog measurements are converted to force values using a resistance-based model. Multiplexing ensures reduced hardware complexity while enabling dual-sensor acquisition.

B. BLE GATT Architecture

The system defines a custom primary service with a single DATA characteristic configured with the Notify property. This characteristic is used to transmit structured protocol frames from the peripheral to the central device in a continuous streaming manner. The DATA characteristic encapsulates all relevant sensor information, including force measurements, sequence number, timestamp, and integrity verification fields, enabling efficient and structured communication over BLE.

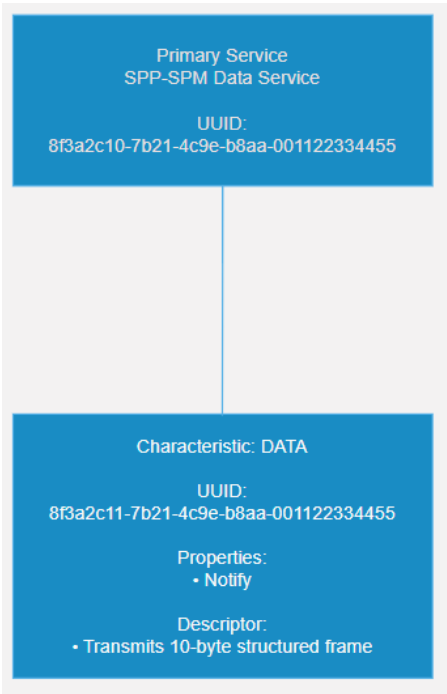


Figura 1. GATT Structure of the Proposed SPP-SPM Service

IV. Custom Protocol Design

The application-layer protocol defines a fixed-length frame compatible with the default BLE MTU (20 bytes). Each frame includes synchronization markers, version control, node identification, sequence numbering, sensor values, timestamp, and CRC.

A. Frame Structure

The frame is defined as a fixed-length structure of 10 bytes, designed to minimize transmission overhead while maintaining data integrity and traceability.

A single-byte header encodes both the protocol version and the message type using a compact 4-bit representation for each field. An 8-bit sequence number enables tracking of transmitted packets and facilitates loss detection at the receiver.

The payload includes two 16-bit force measurements corresponding to the sensors, along with a 16-bit timestamp representing system time in milliseconds using a modular representation.

A CRC-8 (XOR-based) value is appended at the end of the frame to provide integrity verification of the transmitted data.

Table 1: SPP - SPM Protocol Frame Structure

Byte Index	Field Name	Size (bytes)	Type	Description
0	Header	1	uint8	Upper 4 bits: Version (0x01), Lower 4 bits: Message Type (0x01 = DATA)
1	Node ID	1	uint8	Device identifier
2	Sequence Number	1	uint8	Frame counter (Big-endian)
3-4	Force Sensor 1	2	uint16	F1 measurement (Big-endian)
5-6	Force Sensor 2	2	uint16	F2 measurement (Big-endian)
7-8	Force Sensor 3	2	uint16	F3 measurement (Big-endian)
9-10	Force Sensor 4	2	uint16	F4 measurement (Big-endian)
11-12	Force Sensor 5	2	uint14	F5 measurement (Big-endian)
13-14	Force Sensor 6	2	uint16	F6 measurement (Big-endian)
15	Battery Level	1	uint8_t	Battery level measurement
16-17	Timestamp	2	uint16	System time in ms (mod 65536)
18	CRC	1	uint8	XOR checksum

The proposed frame structure measures 19 bytes in total, remaining within the default BLE Maximum Transmission Unit (MTU) of 20 bytes. This design ensures compatibility with standard BLE configurations without requiring MTU negotiation, thereby improving interoperability across different central devices.

Multi-byte fields such as the sequence number, force sensor values, and timestamp are encoded in big-endian format (where the most significant byte comes first). This approach guarantees a consistent bytes order and makes decoding easier across different platforms.

The remaining unused bytes within the default MTU offer room for future protocol extensions, such as additional sensor channels, battery level reporting, error flags, or encryption-related metadata, without needing to redesign the communication layer.

Version Field Definition

In the current implementation, the Version field shall be set to: 0x01.

Devices receiving a DATA frame with an unsupported Version value shall discard the frame.

Future protocol revisions shall increase the Version field while preserving backward compatibility whenever possible.

Node ID Field Definition

The Node ID field uniquely identifies the transmitting peripheral device.

In the current implementation, the Node ID is statically configured to: 0x01.

Future implementations may assign different Node ID values to support multi-node deployments. The valid range for Node ID is 0x00-0xFF.

CRC Definition

The CRC field provides integrity verification of the DATA frame.

The CRC shall be calculated as a cumulative XOR operation over all bytes of the frame, from Byte Index 0 (Header) through Byte Index 8 (Timestamp), excluding the CRC field itself.

Formally:

$$CRC = \bigoplus_{i=0}^8 Byte_i$$

Where:

$Byte_i$ represents the value at byte index i .

The XOR operation is applied sequentially from $i = 0$ to $i = 8$.

Upon reception, the central device shall recompute the CRC using the same procedure and compare it with the received CRC value.

If the CRC validation fail the DATA frame shall be discarded.

V. Implementation

The firmware was implemented using the ESP32 BLE Arduino framework. Custom callback classes manage connection events. A finite state machine controls transmission logic.

A. Communication State Machine

The communication logic of peripheral is modeled as a deterministic Finite State Machine (FSM). The FSM governs frame transmission, acknowledgment handling, and retransmission under timeout conditions.

Defined states

Let:

$S = \{ S_0: Advertising,$

$S_1: Connected,$

$S_2: Transmit_Data \}$

Initial state:

S_0

Transition function

The transition function $\delta: S \times E \rightarrow S$ is defined as follows:

Table 2. State Transition Table of the Communication FSM

Current State	Event	Next State
S_0	Connection established	S_1
S_1	Data ready	S_2
S_2	Frame sent	S_2
S_1	Disconnection	S_0
S_2	Disconnection	S_0

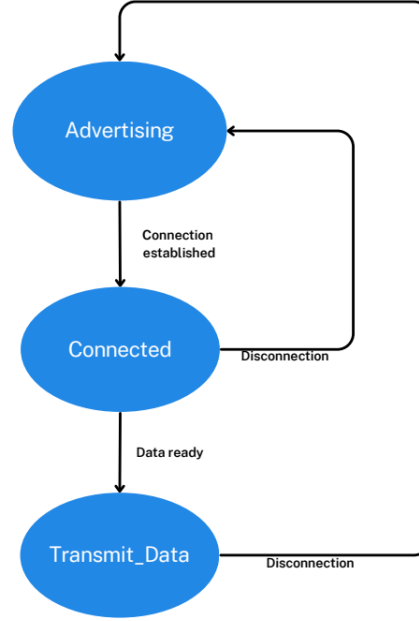


Figura 2. Finite State Machine of the SPP-SPM Communication Protocol

The system follows a continuous streaming communication model once a BLE connection is established. Sensor data is periodically acquired and encapsulated into fixed-length DATA frames, which are transmitted without interruption.

If a disconnection occurs, the system automatically transitions to the Advertising state, enabling reconnection and restoring normal operation.

VI. Experimental Validation

A. Experimental Setup

The experimental evaluation was conducted using an ESP32-C3 microcontroller configured as a BLE peripheral implementing the proposed SPP-SPM custom service- Two force-sensitive resistors (FSRs) were connected through a digital multiplexing scheme controlled by GPIO pins. Analog readings were acquired using internal 12-bit ADC.

The BLE device operated using the default MTU size (20 bytes) without extended MTU negotiation. The DATA characteristics were configured with the Notify property.

The central device used for testing was a computer running a custom Python-based BLE client responsible for receiving notifications.

B. Results and Analysis

The experimental evaluation focused primarily on validating the functional correctness and reliability of the proposed protocol rather than performing extensive statistical benchmarking.

Overall, the experimental results demonstrate that the proposed protocol successfully achieves deterministic application-layer reliability while maintaining compatibility with the default BLE MTU configuration.

C. Comparison with Standard BLE Notification Mechanism

Standard BLE notifications provide an efficient mechanism for transmitting data from a peripheral to a central without requiring explicit acknowledgement at the application layer. Although reliability is handled at the Link Layer through automatic retransmissions, successful delivery is not explicitly confirmed at the application level [1].

In many BLE-based sensor systems, data is transmitted periodically without additional structure, such as sequence numbering or integrity verification beyond the Link Layer. While this approach minimizes overhead and simplifies implementation, it may introduce limitations in applications requiring data tracking, validation, or post-processing analysis.

The proposed SPP-SPM protocol builds upon the native BLE notification mechanism by incorporating a lightweight application-layer structure. This includes sequence numbering for frame tracking, a compact header for protocol identification, and a CRC field for additional data integrity verification.

Unlike native BLE notifications, the proposed approach enables:

- Frame ordering and loss detection through sequence numbers.
- Additional integrity validation using lightweight XOR-based CRC.
- Structured and deterministic data logging.

This design maintains the low overhead and efficiency of BLE notifications while enhancing traceability and robustness at the application layer.

The protocol is implemented using a fixed-length frame of 10 bytes, ensuring compatibility with the default 20-byte MTU and avoiding fragmentation.

Table 3. Comparison Between Native BLE and Proposed Protocol

Feature	Native BLE notify	Proposed SPP-SPM Protocol
Application ACK	No	No
Sequence Number	No	Yes
Retransmission Control	Link-layer only	Link-layer only
Deterministic Logging	Limited	Improved (via sequencing)
Data Integrity (Application)	No	Yes (CRC)
MTU Compatibility	Yes	Yes
Overhead	Minimal	Low (10-byte frame)

VII. Limitations

The current implementation operates under BLE MTU constraints (20 bytes) without extended MTU negotiation. Although the proposed frame size (10 bytes) ensures full compatibility, larger payloads or future system extensions may require MTU optimization.

Reliability is delegated to the BLE protocol stack, which provides error detection and retransmission mechanisms at the Link Layer. As a result, no additional reliability mechanisms are implemented at the application layer. While this simplifies the system and reduces overhead, it does not provide explicit confirmation of frame delivery at the application level.

The system has been validated under short-range indoor conditions and has not been characterized under high-interference or long-range environments. Therefore, performance under challenging wireless conditions remains an open area for further evaluation.

VIII. Conclusions

The proposed SPP-SPM protocol demonstrates that an efficient and structured data transmission model can be achieved over BLE notifications with minimal overhead and without requiring MTU negotiation.

By leveraging the inherent reliability mechanisms provided by the BLE Link Layer, the protocol avoids redundant application-layer reliability features, resulting in a simplified and low-latency communication system.

The integration of sequence numbering enables frame tracking and loss detection, while the inclusion of a lightweight CRC provides an additional layer of data integrity verification. Furthermore, the use of a compact fixed-length frame (10 bytes) ensures efficient bandwidth utilization and full compatibility with standard BLE configurations.

Overall, the proposed approach achieves a balance between efficiency, simplicity, and robustness, making it suitable for real-time embedded sensing applications where low overhead and continuous data streaming are required.

IX. References

- [1] Bluetooth Special Interest Group, Bluetooth Core Specification Version 5.4, Bluetooth SIG, 2023.
- [2] M. A. Al-Shareeda, M. A. Saare, S. Manickam y S. Karuppayah, «Bluetooth Low Energy for Internet of Things: Review, Challenges, and Open Issues,» *Indonesian Journal of Electrical*

Engineering and Computer Science, vol. 31, nº 2, pp. 1182-1189, 2023.

- [3] M. Siekkinen, M. Hienkari, J. K. Nurminen y J. Nieminen, «How low energy is Bluetooth Low Energy? Comparative measurements with ZigBee/802.15.4,» de *IEEE Wireless Communications and Networking Conference (WCNC)*, Paris, 2012.
- [4] L. L. e. al., «A Runtime Admission Control for Industrial IoT over Bluetooth Low Energy,» *Journal of Network and Computer Applications*, 2025.